```python
#NAME : Shivam Bhalgama
#Roll No : 24BCE373
#PRECTICAL_1:BASIC OF PYTHON,NUMPY,PANDAS,MATPLOTLIB
#DATE:06-02-2025
#SUBLECT:AI &ML

#python basic

print("hello world")
#input statment
a=int(input("enter the number:"))
print(a)
```

```
hello world

enter the number: 345

345
```

```python
#varible decleration
a=5
b="Shivam"
c=10.2
print(a)
print(b)
print(c)
print(type(a))
print(a,b,c)
print("sum is=",a+c)
```

```
5
Shivam
10.2
<class 'int'>
5 Shivam 10.2
sum is= 15.2
```

```python
#data type in python

#string
str='Shivam'
print(str)
print(str[0])
print(str[1:3])
print(str[2:])
print(str+" Bhalgama")
print(str*4)
print(str[-1])
print(str[-3:-1])
```

```
Shivam
S
hi
ivam
Shivam Bhalgama
ShivamShivamShivamShivam
m
va
```

```python
#list
list=['qwe',123,'dgfh',534,45.4]
mlist=['sdg',354]
print(list)
print(list[0])
print(list[2:3])
print(list[3:])
print(list+mlist)
print(mlist*2)
```

```
['qwe', 123, 'dgfh', 534, 45.4]
qwe
['dgfh']
[534, 45.4]
['qwe', 123, 'dgfh', 534, 45.4, 'sdg', 354]
['sdg', 354, 'sdg', 354]
```

```python
#tuple
tuple=('qwe',123,'dgfh',534,45.4)
mtuple=('sdg',354)
print(tuple)
print(tuple[0])
print(tuple[2:3])
print(tuple[3:])
print(tuple+mtuple)
print(mtuple*2)
```

```
('qwe', 123, 'dgfh', 534, 45.4)
qwe
('dgfh',)
(534, 45.4)
('qwe', 123, 'dgfh', 534, 45.4, 'sdg', 354)
('sdg', 354, 'sdg', 354)
```

```python
#dictionary
dict={}
dict['one']=50
dict[500]='tgdh'
mdict={'sdg':564,'qwer':'fjhgj',354:3456}
print(dict)
print(dict['one'])
```

```
print(mdict['qwer'])
print(mdict.keys())
print(mdict.values())
```

```
{'one': 50, 500: 'tgdh'}
50
fjhgj
dict_keys(['sdg', 'qwer', 354])
dict_values([564, 'fjhgj', 3456])
```

```
#oprator in python:1.arithmetic 2.relation 3.logical 4.assement
5.bitwise
#question:two list deffine and apply opration opreter
list=[1,2,3,4,5]
alist=[354,4543,53,2232,5]
print(list[0]+alist[3])
print(list[0]-alist[3])
print(list[0]/alist[3])
print(list[0]%alist[3])
print(list[0]*alist[3])
print(list[0]<alist[3])
print(list[0]<=alist[3])
print(list[0]>alist[3])
print(list[0]!=alist[3])
print(list[0]>=alist[3])
print(list[0]|alist[3])
print(list[0]^alist[3])
print(list[0]&alist[3])
print(list[0]==alist[3])
blist=[list[0]+alist[0],list[1]+alist[1],list[2]+alist[2],list[3]+alis
t[3],list[4]+alist[4]]
```

```
2233
-2231
0.00044802867383512545
1
2232
True
True
False
True
False
2233
2233
0
False
```

```
#condition statment
```

```python
#question:chack aligibility of age for voting
age=int(input("enter the your age="))
if (age>=18):
    print("you are aligible for voting")
else:
    print("you are not aligible for voting")
```

```
enter the your age= 23
```

```
you are aligible for voting
```

```python
#question:odd and even number
n=int(input("entre the number="))
if(n%2==0):
    print("even")
else:
    print("odd")
```

```
entre the number= 89
```

```
odd
```

```python
#question:gretest number of given three number
a=10
b=5
c=3
if a>b:
    if a>c:
        print("a is gretest")
    else:
        print("c is gretest")
elif b>c:
    print("b is gretest")
else:
    print("c is gretest")
```

```
a is gretest
```

```python
#loop 1.while 2.for
```

```python
#question:print 1 to 10 number
i=1
while i<=10:
    print(i)
    i=i+1
```

```
1
2
3
4
5
```

```
6
7
8
9
10
```

```
n=int(input("enter the number="))
i=1
while i<=10:
    print(n,"*",i,"=",n*i)
    i=i+1
```

```
enter the number= 6
```

```
6 * 1 = 6
6 * 2 = 12
6 * 3 = 18
6 * 4 = 24
6 * 5 = 30
6 * 6 = 36
6 * 7 = 42
6 * 8 = 48
6 * 9 = 54
6 * 10 = 60
```

```
n1=0
n2=1
next=n1+n2
print(0,1)
n=int(input("enter the number="))
for i in range(3,n+1):
    n1=n2
    n2=next
    next=n1+n2
    print(next)
```

```
0 1
```

```
enter the number= 5
```

```
2
3
5
```

```
n=int(input("enter the number="))
fact=1
for i in range(1,n+1):
    fact=fact*i
print(fact)
```

```
enter the number= 8
```

```
40320
```

```
#function

t=0
def multi(arg1,arg2):
    t=arg1*arg2  #locaL VARIBLE
    print(t)
multi(10,20)
print(t)  #global varible
```

```
200
0
```

```
#crete a function of n number of sum
def sum(n):
    sum2=0
    for i in range(1,n+1):
        sum2=sum2+i
    print(sum2)


sum(10)
```

```
55
```

```
#Numpy


import numpy as np
a=np.array([1,2,3,4],float)
a
a[:2]
a[3]
```

```
np.float64(4.0)
```

```
#2D array
import numpy as np
b=np.array([[1,2,3],[4,5,6]])
b[0,0]
```

```
np.int64(1)
```

```
b.shape
```

```
(2, 3)
```

```
import numpy as np
print(b.dtype)
print(len(b))
```

```
b=b.reshape(3,2)
print(b)
b.transpose()
print(b)
b.flatten()
print(b)
```

```
int64
2
[[1 2]
 [3 4]
 [5 6]]
[[1 2]
 [3 4]
 [5 6]]
[[1 2]
 [3 4]
 [5 6]]
```

```
import numpy as np
x=np.array([9,8],int)
y=np.array([7,6],int)
z=np.array([5,4],int)
a=np.concatenate((x,y,z))
print(a)
```

```
[9 8 7 6 5 4]
```

```
# concatenate() function
g=np.array([[1,2,3],[4,5,6]],float)
h=np.array([[7,8,9],[10,11,12]],float)
np.concatenate((g,h))
a=np.concatenate((g,h),axis=0)
```

```
a=np.concatenate((g,h),axis=0)


np.concatenate((g,h),axis=1)
```

```
array([[ 1.,  2.,  3.,  7.,  8.,  9.],
       [ 4.,  5.,  6., 10., 11., 12.]])
```

```
np.arange(1,11,2,dtype=float)
```

```
array([1., 3., 5., 7., 9.])
```

```
np.ones((5,2),dtype=int)
```

```
array([[1, 1],
       [1, 1],
       [1, 1],
```

```
       [1, 1],
       [1, 1]])
```

```
np.zeros((2,3),dtype=float)
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
arr1=np.array([[1,2,3],[4,5,6]],float)
arr2=np.array([[3,4,6],[4,6,9]],float)
arr1*arr2
array1=np.array([[1,2,3],[4,5,6]],float)
array2=np.array([3,4,6],float)
array1-array2
```

```
array([[-2., -2., -3.],
       [ 1.,  1.,  0.]])
```

```
l=np.ones((3,3),float)
k=np.array([-3,4,2],float)
l+k[np.newaxis,:]
```

```
array([[-2.,  5.,  3.],
       [-2.,  5.,  3.],
       [-2.,  5.,  3.]])
```

```
a=np.array([1,16,25],float)
np.sqrt(a)
```

```
array([1., 4., 5.])
```

```
arr=np.array([1.2,3.4,5.7,7.8,6.5],float)
np.ceil(arr)
```

```
array([2., 4., 6., 8., 7.])
```

```
np.rint(arr)
```

```
array([1., 3., 6., 8., 6.])
```

```
m=np.array([1,2,3,4],float)
for i in m:
    print(i)
```

```
1.0
2.0
3.0
4.0
```

```
m=np.array([[20,30],[10,40],[50,60]])
for (a,b) in m:
     print(a*b)
```

```
600
400
3000
```

```
mat=np.array([[1,2,3],[4,5,6]],float)
np.mean(mat,axis=0)
```

```
array([2.5, 3.5, 4.5])
```

```
np.mean(mat,axis=1)
```

```
array([2., 5.])
```

```
b=np.array([1,1,2,3,4,4,4,5,5,5,5,7,78,30])
np.unique(b)
```

```
array([ 1,  2,  3,  4,  5,  7, 30, 78])
```

```
mat1=np.array([1,2,3,4,5,6])
mat2=np.array([18,17,2,15,1,13])
mat1<mat2
```

```
array([ True,  True, False,  True, False,  True])
```

```
m=np.array([[30,20,50],[70,40,100],[90,80,60]])
np.where(m<=60,m,m/10)
```

```
array([[30., 20., 50.],
       [ 7., 40., 10.],
       [ 9.,  8., 60.]])
```

```
o=np.array([14,np.NaN,43,12,np.Inf],float)
o
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[50], line 1
----> 1 o=np.array([14,np.NaN,43,12,np.Inf],float)
      2 o

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\numpy\__init__.py:413, in __getattr__(attr)
    410     raise AttributeError(__former_attrs__[attr], name=None)
    412 if attr in __expired_attributes__:
--> 413     raise AttributeError(
    414         f"`np.{attr}` was removed in the NumPy 2.0 release. "
    415         f"{__expired_attributes__[attr]}",
    416         name=None
    417     )
    419 if attr == "chararray":
```

```
    420     warnings.warn(
    421         "`np.chararray` is deprecated and will be removed from
"
    422         "the main namespace in the future. Use an array with a
string "
    423         "or bytes dtype instead.", DeprecationWarning,
stacklevel=2)

AttributeError: `np.NaN` was removed in the NumPy 2.0 release. Use
`np.nan` instead.
```

```
# only finite() function
np.isfinite(o)

m=float(input("Enter a number for comparison:"))
n=np.array([[1,2,3],[4,5,6],[7,8,9]],float)
sel=(m>=n)
n[sel]
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call
last)
Cell In[49], line 2
      1 # only finite() function
----> 2 np.isfinite(o)
      4 m=float(input("Enter a number for comparison:"))
      5 n=np.array([[1,2,3],[4,5,6],[7,8,9]],float)

NameError: name 'o' is not defined
```

```
# dot product for matric or say vectors
import numpy as np
a=np.array([[1,2],[4,5]],float)
b=np.array([[11,12],[14,15]],float)
c=np.array([3,4],float)
np.dot(a,b)
```

```
array([[ 39.,  42.],
       [114., 123.]])
```

```
# dot product
np.dot(a,c)
```

```
array([11., 32.])
```

```
# cross product for matrix or say vectors
a=np.array([1,2,3],float)
b=np.array([4,5,6],float)
np.cross(a,b)
```

```
array([-3.,  6., -3.])
```

```
np.random.rand(10)
```

```
array([0.05338744, 0.92219351, 0.34927186, 0.29961574, 0.97420908,
       0.51527336, 0.20400242, 0.73451218, 0.6226253 , 0.83582812])
```

```
np.random.rand(3,3)
```

```
array([[0.57057551, 0.94626787, 0.92538957],
       [0.00098295, 0.43604544, 0.41515031],
       [0.4906047 , 0.86531539, 0.39157492]])
```

```
# generation of a single random number
np.random.random(1)
```

```
array([0.22442877])
```

```
# generation of random number of integer data type from [min,max]
#provided area
np.random.randint(100,1000)
```

```
276
```

```
# shuffling the elements of array through random() function
c=np.arange(1,11)
np.random.shuffle(c)
c
```

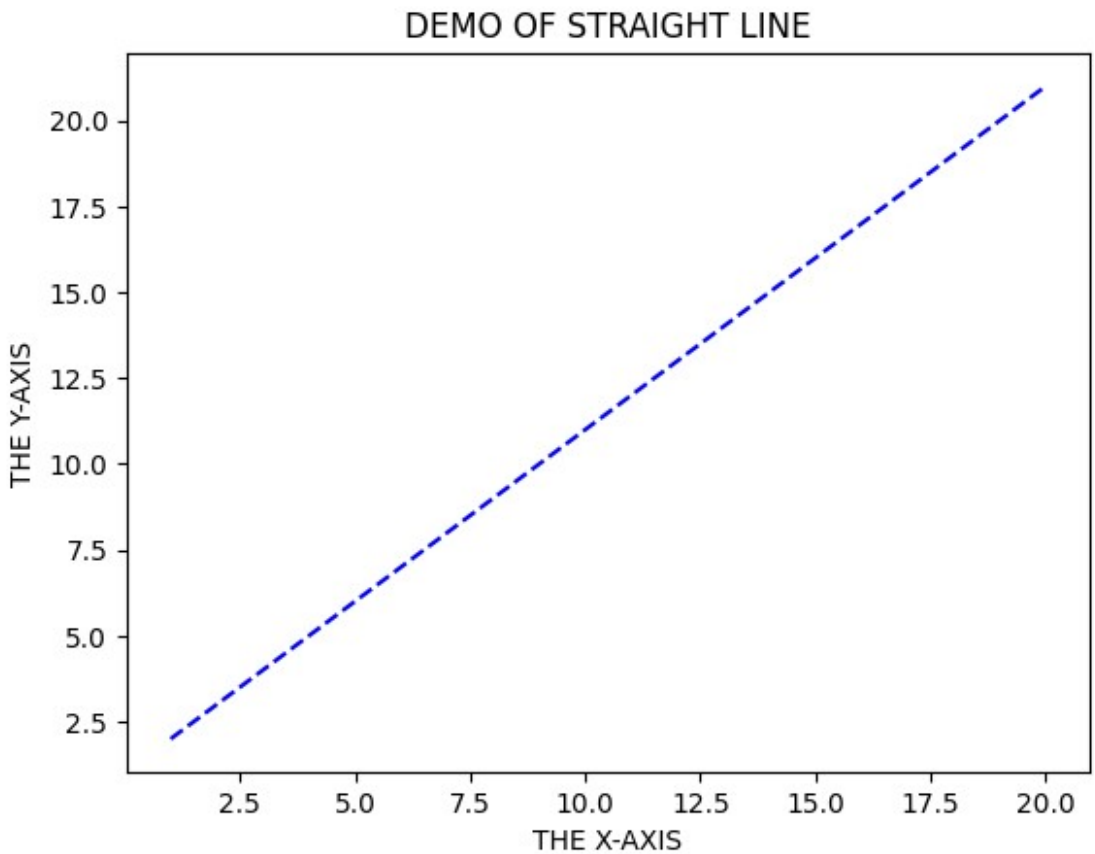
```
array([ 6,  4,  9,  1,  8,  7, 10,  3,  5,  2])
```

```
m=np.arange(1,20)
np.save('outfile',m)
m=np.load('outfile.npy')
print(m)
```

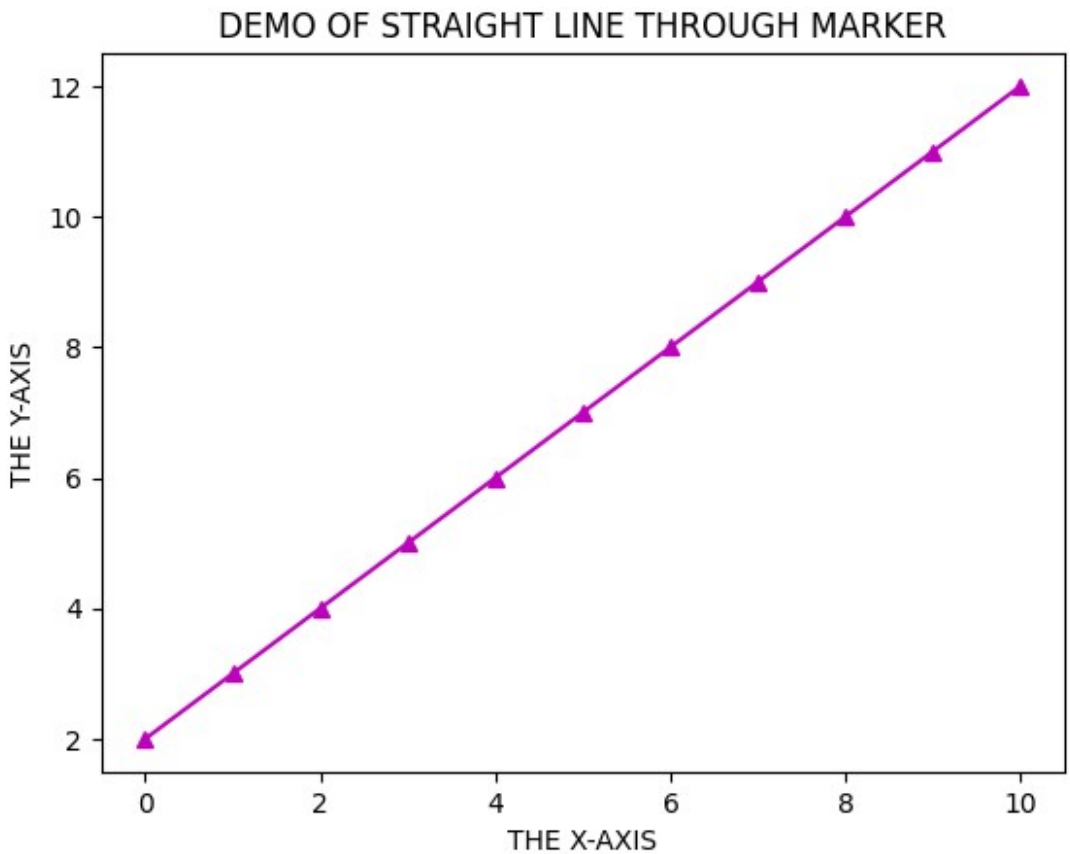
```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
```

```
#matplotlib
```

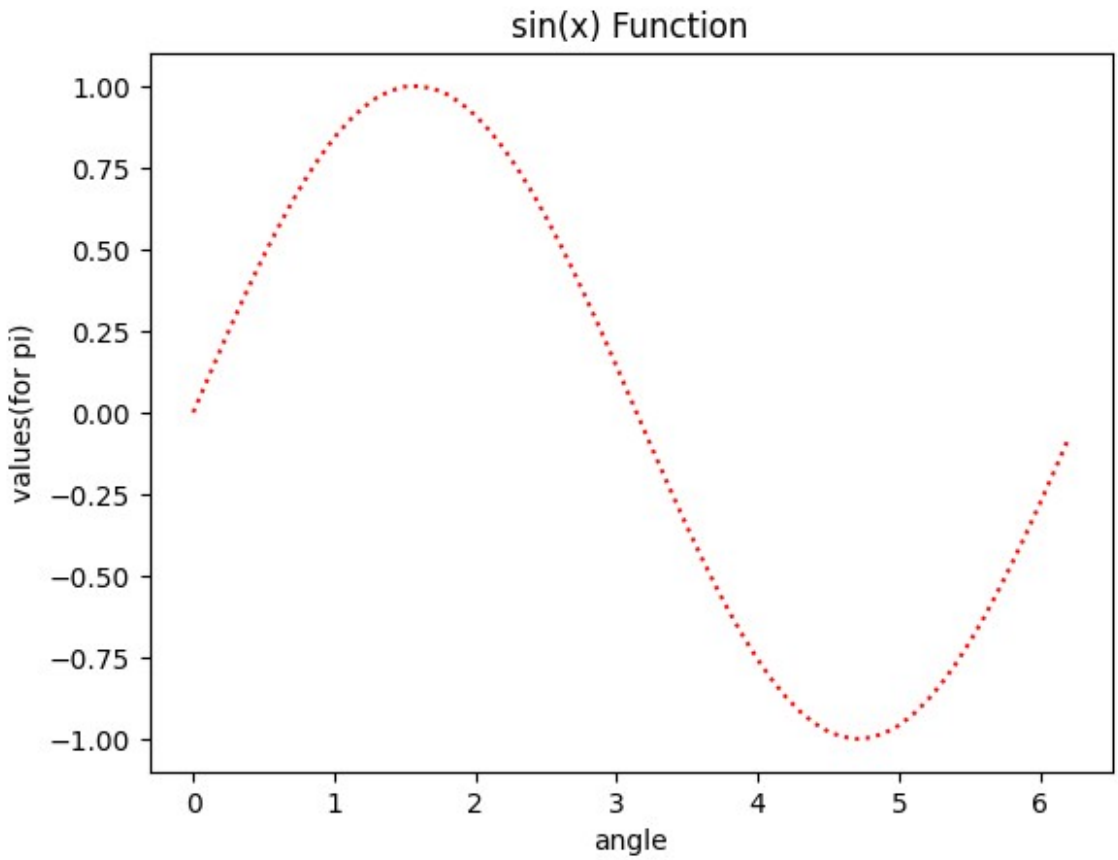
```
import matplotlib.pyplot as plt
import numpy as np
# Straight line plotting
x=np.arange(1,21)
y=x+1
plt.ylabel("THE Y-AXIS")
plt.xlabel("THE X-AXIS")
plt.title("DEMO OF STRAIGHT LINE")
plt.plot(x,y,linestyle="--",color="b")
plt.show()
```



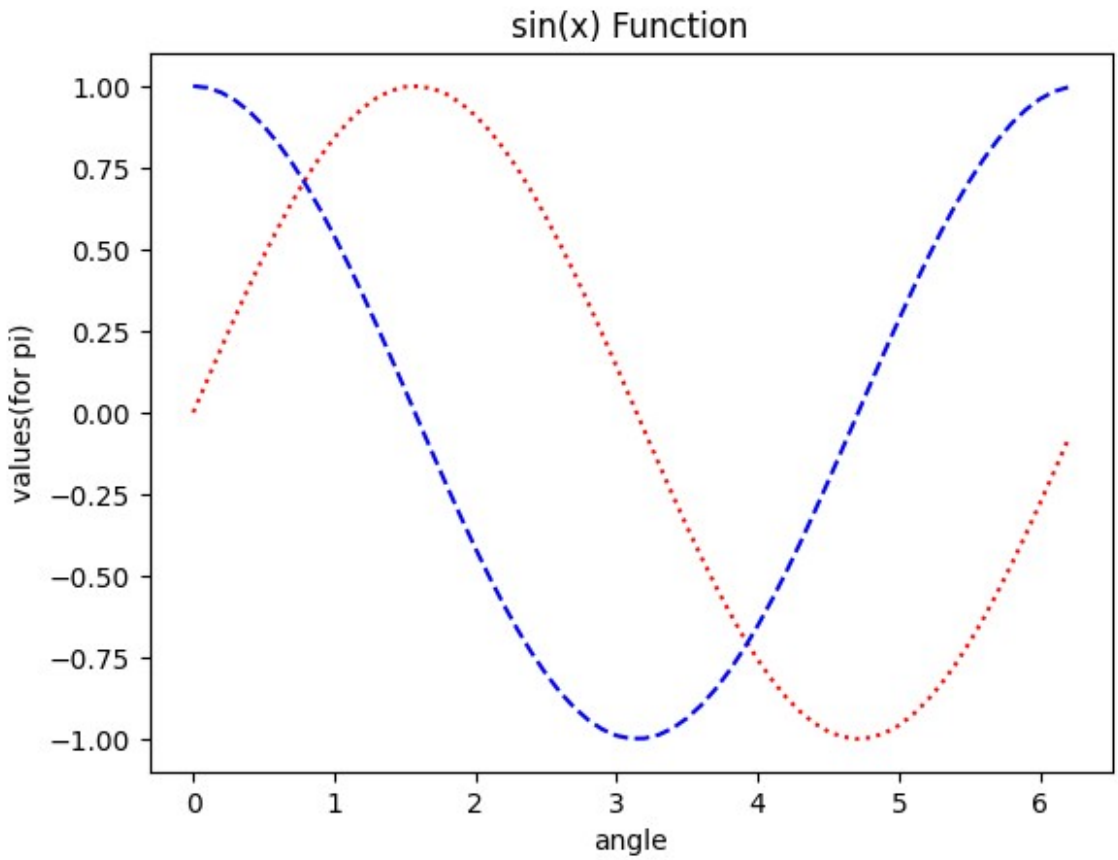

```
# straight line plotting through marker option
x=np.arange(0,11)
y=x+2
plt.title("DEMO OF STRAIGHT LINE THROUGH MARKER")
plt.xlabel("THE X-AXIS")
plt.ylabel("THE Y-AXIS")
plt.plot(x,y,color="m",marker="^")
plt.show()
```



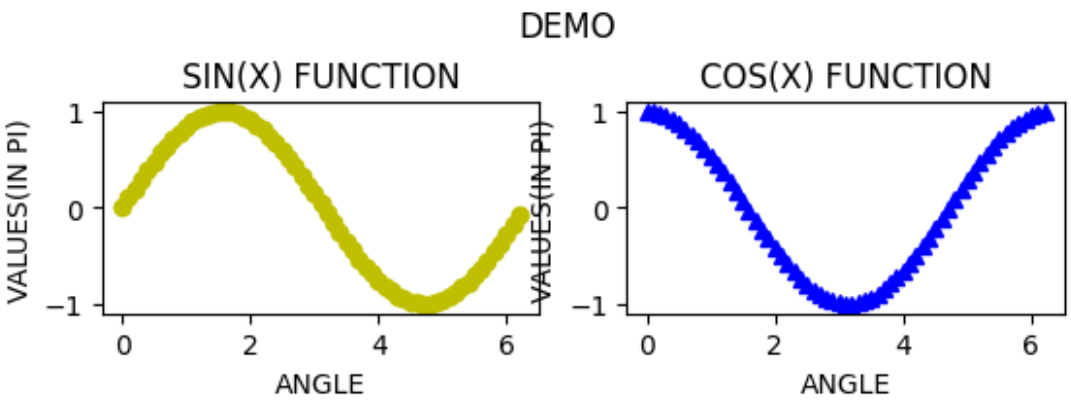

```
x=np.arange(0,2*np.pi,0.1)
y=np.sin(x)
plt.title("sin(x) Function")
plt.xlabel("angle")
plt.ylabel("values(for pi)")
plt.plot(x,y,linestyle=":",color="r")
plt.show()
```



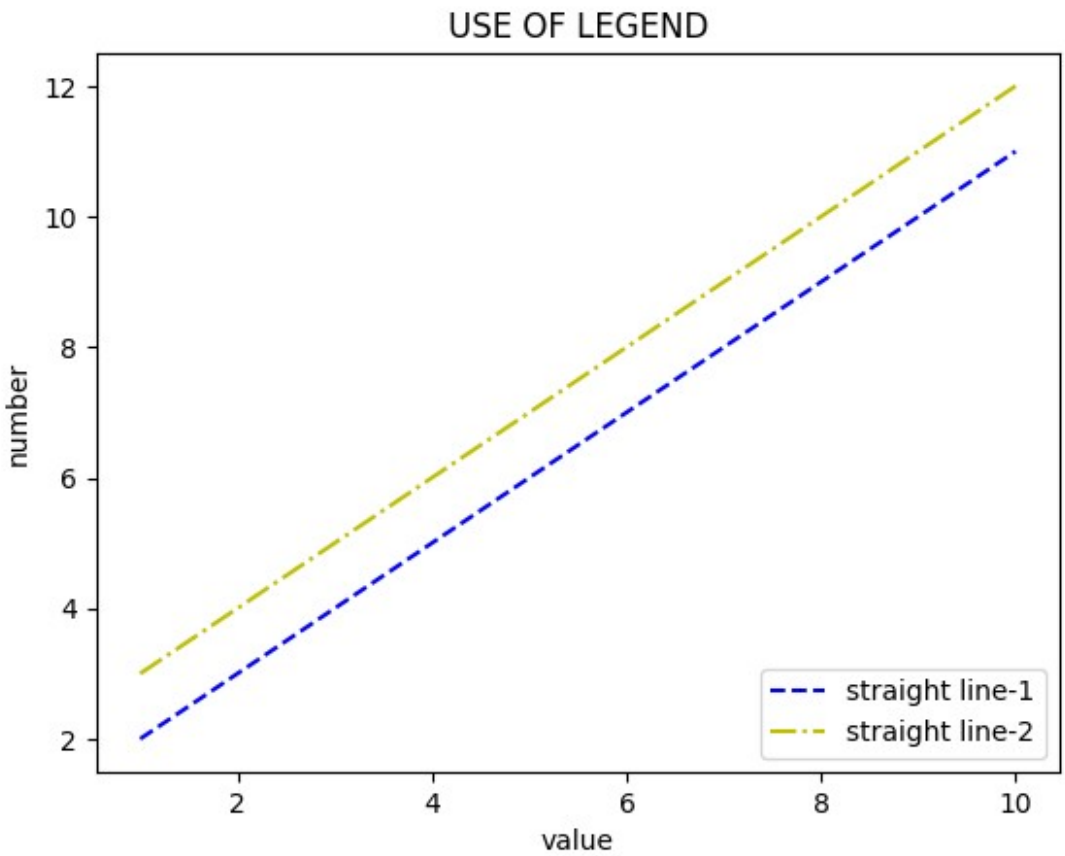

```
x=np.arange(0,2*np.pi,0.1)
y=np.sin(x)
z=np.cos(x)
plt.title("sin(x) Function")
plt.xlabel("angle")
plt.ylabel("values(for pi)")
plt.plot(x,y,linestyle=":",color="r")
plt.plot(x,z,linestyle="--",color="b")
plt.show()
```



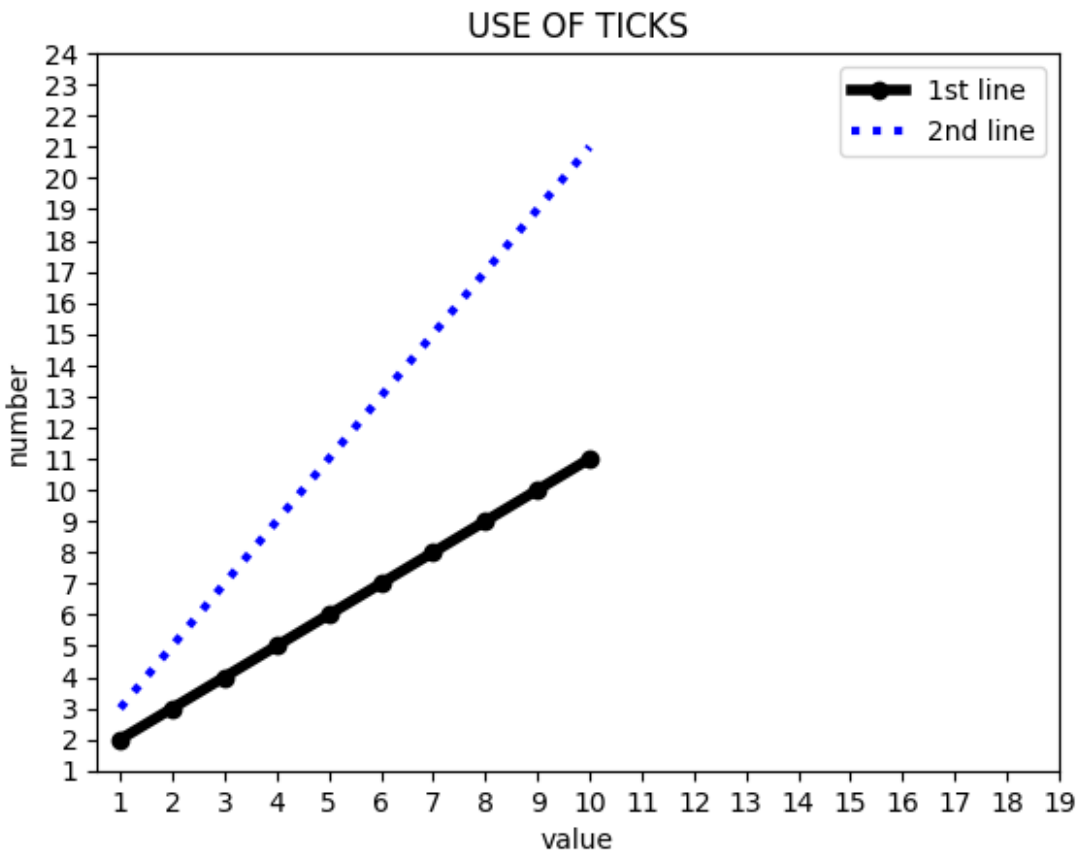

```
x=np.arange(0,2*np.pi,0.1)
y1=np.sin(x)
y2=np.cos(x)
plt.suptitle("DEMO")
plt.subplot(3,2,1)
plt.title("SIN(X) FUNCTION")
plt.xlabel("ANGLE")
plt.ylabel("VALUES(IN PI)")
plt.plot(x,y1,marker="o",color="y")
plt.subplot(3,2,2)
plt.title("COS(X) FUNCTION")
plt.xlabel("ANGLE")
plt.ylabel("VALUES(IN PI)")
plt.plot(x,y2,marker="^",color="b")
plt.show()
```



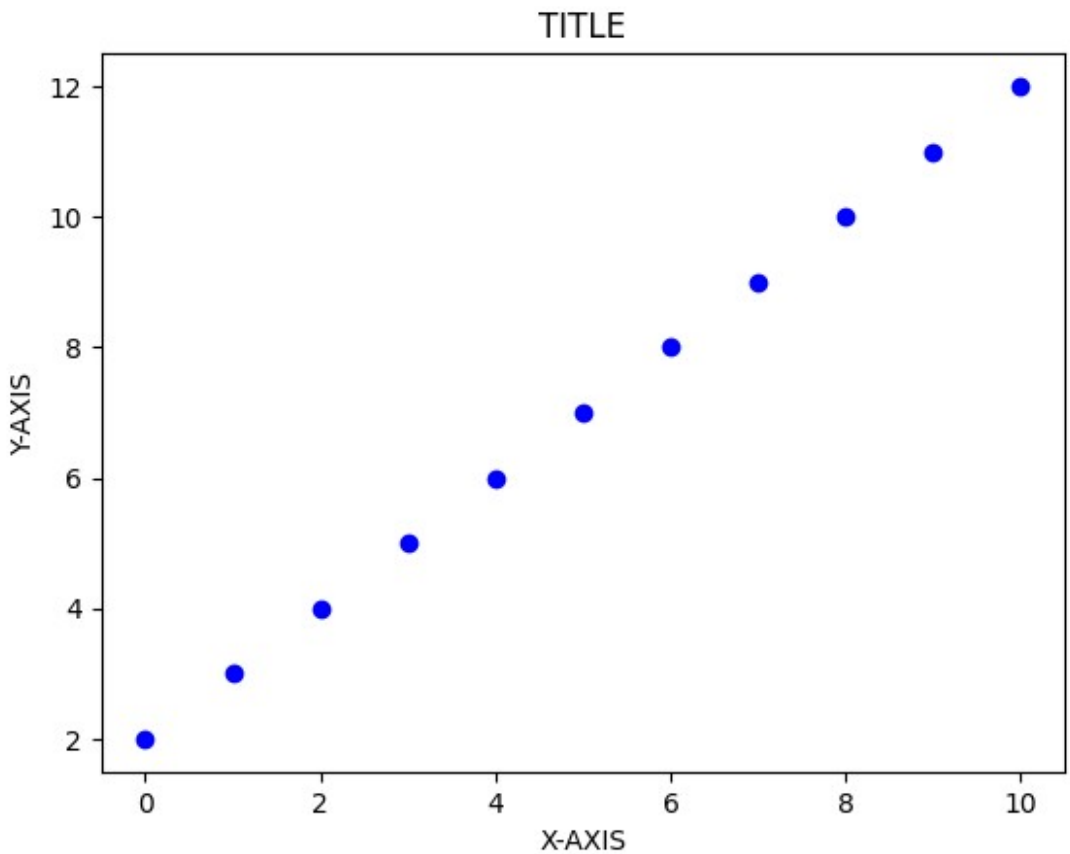

```
x=np.arange(1,11)
y1=x+1
y2=x+2
plt.title("USE OF LEGEND")
plt.xlabel("value")
plt.ylabel("number")
plt.plot(x,y1,linestyle="--",color="b",label="straight line-1")
plt.plot(x,y2,linestyle="-.",color="y",label="straight line-2")
plt.legend(loc=4)
plt.show()
```



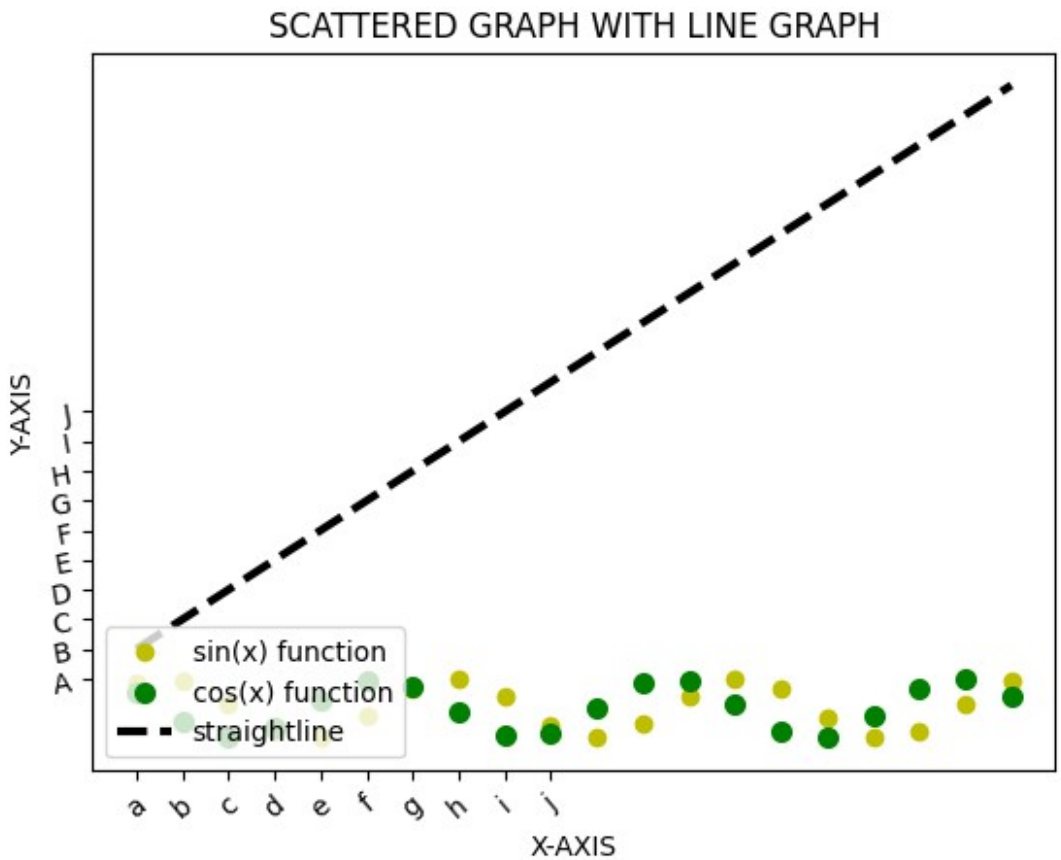
```
x=np.arange(1,11)
y1=x+1
y2=2*x+1
plt.title("USE OF TICKS")
plt.xlabel("value")
plt.ylabel("number")
plt.plot(x,y1,marker="o",linewidth="4",color="k",label="1st line")
plt.plot(x,y2,linestyle=":",linewidth="3",color="b",label="2nd line")
plt.legend(loc=1)
plt.xticks(np.arange(1,20))
plt.yticks(np.arange(1,25))
plt.show()
```



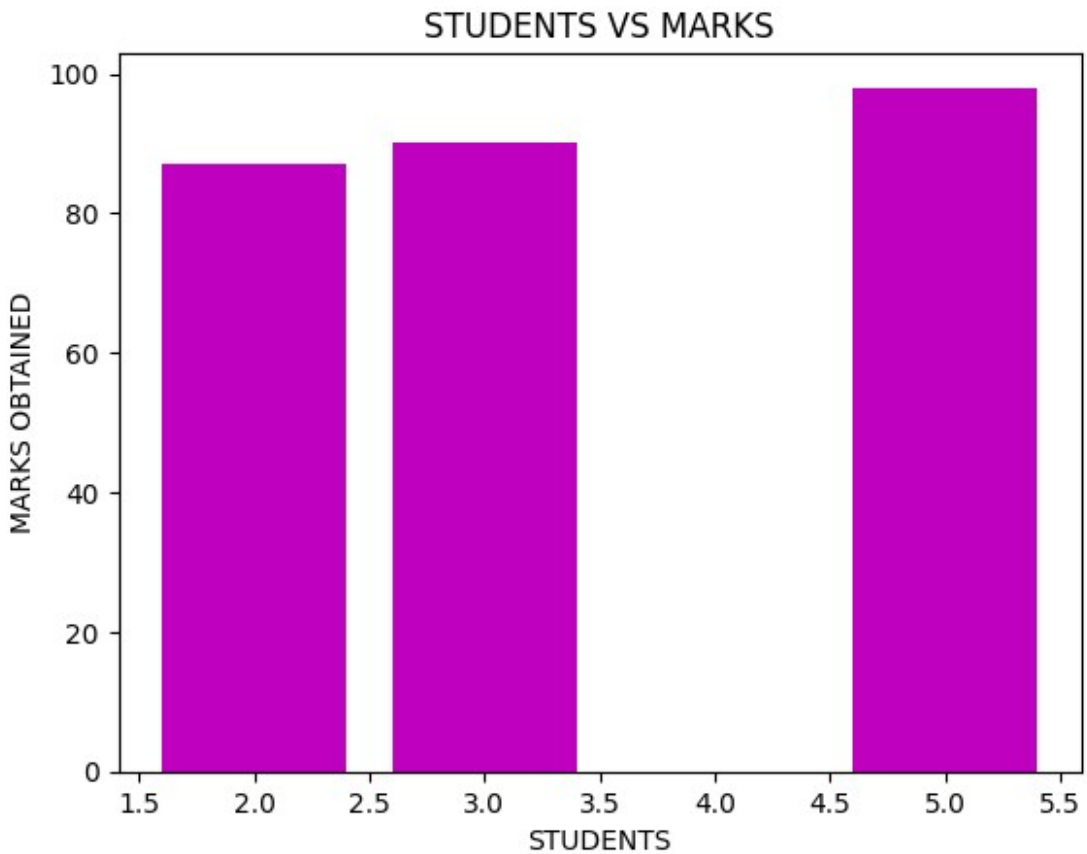

```
x=np.arange(11)
y=x+2
plt.title("TITLE")
plt.xlabel("X-AXIS")
plt.ylabel("Y-AXIS")
plt.plot(x,y,marker="o",color="b",linestyle="")
plt.show()
```



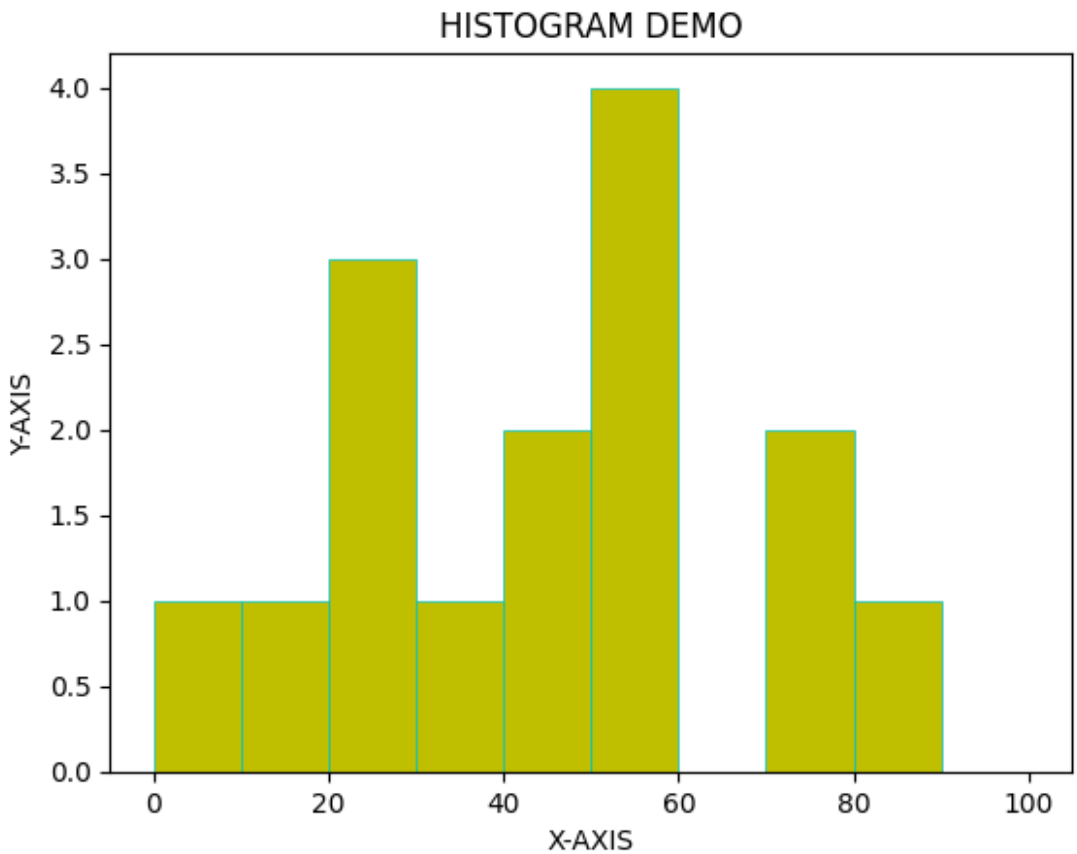

```
x=np.arange(1,21)
y1=np.sin(x)
y2=np.cos(x)
y3=x+1
plt.title("SCATTERED GRAPH WITH LINE GRAPH")
plt.xlabel("X-AXIS")
plt.ylabel("Y-AXIS")
plt.scatter(x,y1,linewidth=1,color="y",label="sin(x) function")
plt.scatter(x,y2,linewidth=2,color="g",label="cos(x) function")
plt.plot(x,y3,linestyle="--",color="k",linewidth=3,label="straightline
")
plt.legend(loc=3)
plt.xticks(np.arange(1,11),
['a','b','c','d','e','f','g','h','i','j'],rotation=40)
plt.yticks(np.arange(1,11),
['A','B','C','D','E','F','G','H','I','J'],rotation=10)
plt.show()
```




```
# no of students vs marks obtained through bar graph
a=[5,3,2]
b=[98,90,87]
plt.title("STUDENTS VS MARKS")
plt.xlabel("STUDENTS")
plt.ylabel("MARKS OBTAINED")
plt.bar(a,b,color="m")
plt.show()
```




```
# purshape of product at shops vs the for maximum sale from that
shopsthrough histogram
a=np.array([22,87,5,43,56,73,55,54,11,20,51,43,79,31,27])
plt.title("HISTOGRAM DEMO")
plt.xlabel("X-AXIS")
plt.ylabel("Y-AXIS")
plt.hist(a,bins=[0,10,20,30,40,50,60,70,80,90,100],color="y",edgecolor
="c",linewidth=0.5)
plt.show()
```




```
a=10
a
```

```
10
```

```
#pandas

import pandas as pd
import numpy as np
a=np.array([1,2,3,4,5])
s=pd.Series(a)
print(s)
print(s[1])
print('\n')
print(s[3:4])
s[:3]
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
2
```

```
3    4
dtype: int64
```

```
0    1
1    2
2    3
dtype: int64
```

```
import pandas as pd
import numpy as np
a=np.array(['a','b','c','d','e'])
s=pd.Series(a,index=['first','second','third','fourth','ftfth'])
print(s)
print("\nindexes in series are :: ")
print(s.index)
```

```
import pandas as pd
import numpy as np
a=np.array([1,2,3,4,5])
s=pd.Series(a,index=['a','b','c','d','e'])
print(s)
print(s[1:5:2])
print(s[0:6:2])
```

```
import pandas as pd
df=pd.DataFrame()
print(df)
```

```
import pandas as pd
s=pd.Series(['a','b','c','d','e'])
df=pd.DataFrame(s)
print(df)
```

```
import pandas as pd
name=pd.Series(['Hardik','Virat'])
team=pd.Series(['MI','RCB'])
dic={'name':name,'team':team}
df=pd.DataFrame(dic)
print(df)
```

```
import pandas as pd
l=[{"Name":'Sachin','SirName':'Bhardwaj'},
   {"Name":'Vinod','SirName':'Verma'},
   {"Name":'Rajesh','SirName':'MIshara'}]
df1=pd.DataFrame(l)
print(df1)
```

```
import pandas as pd
l=[{"Name":'Sachin','SirName':'Bhardwaj'},
```

```
   {"Name":'Vinod','SirName':'Verma'}]
df1=pd.DataFrame(l)
print(df1)
for(row_index,row_value) in df1.iterrows():
    print('\n Row index is :: ',row_index)
    print('Row value is ::')
    print(row_value)
```

```
import pandas as pd
empdata={'empid':[101,102,103,104,105,106],
         'ename':['shivam','khus','jiyansh','zahir','vedant','prins'],
         'doj':['12-01-2022','1-01-2022','2-08-2012','12-08-2002','2-
05-2003','12-01-2022']
        }
df=pd.DataFrame(empdata)
print(df)
```

```
   empid    ename         doj
0    101   shivam  12-01-2022
1    102     khus   1-01-2022
2    103  jiyansh   2-08-2012
3    104    zahir  12-08-2002
4    105   vedant   2-05-2003
5    106    prins  12-01-2022
```

```
df.empid
```

```
0    101
1    102
2    103
3    104
4    105
5    106
Name: empid, dtype: int64
```

```
df[['empid','ename']]
```

```
   empid    ename
0    101   shivam
1    102     khus
2    103  jiyansh
3    104    zahir
4    105   vedant
5    106    prins
```

```
import pandas as pd
s=pd.Series([1,2,3,4])
df=pd.DataFrame(s)
df.columns=['list1']
df['list2']=20
```

```
df['list3']=df['list2']+df['list1']
print(df)
```

```
   list1  list2  list3
0      1     20     21
1      2     20     22
2      3     20     23
3      4     20     24
```

```
import pandas as pd
s=pd.Series([1,2,3,4])
df=pd.DataFrame(s)
df.columns=['list1']
df['list2']=40
df1=df.drop('list2',axis=1)
df2=df.drop(index=[2,3],axis=0)
print(df)
print("Aftre deletion :: ")
print(df1)
print("Aftre row deletion :: ")
print(df2)
```

```
   list1  list2
0      1     40
1      2     40
2      3     40
3      4     40
Aftre deletion ::
   list1
0      1
1      2
2      3
3      4
Aftre row deletion ::
   list1  list2
0      1     40
1      2     40
```

```
import pandas as pd
runs={'TCS':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000},
      'WIPRO':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000},
      'L&T':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000}}

df=pd.DataFrame(runs)
print(df)
print(df.loc['qtr3':])
print(df.loc['qtr1':'qtr3',:])
```

```
       TCS  WIPRO   L&T
qtr1  2500   2500  2500
```

```
qtr2  2000   2000  2000
qtr3  3000   3000  3000
qtr4  2000   2000  2000
       TCS  WIPRO   L&T
qtr3  3000   3000  3000
qtr4  2000   2000  2000
       TCS  WIPRO   L&T
qtr1  2500   2500  2500
qtr2  2000   2000  2000
qtr3  3000   3000  3000
```

```
import pandas as pd
runs={'TCS':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000},
      'WIPRO':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000},
      'L&T':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000}}

df=pd.DataFrame(runs)
print(df)
print(df.loc[:,'TCS'])
print(df.loc[:,'TCS':"WIPRO"])
```

```
       TCS  WIPRO   L&T
qtr1  2500   2500  2500
qtr2  2000   2000  2000
qtr3  3000   3000  3000
qtr4  2000   2000  2000
qtr1    2500
qtr2    2000
qtr3    3000
qtr4    2000
Name: TCS, dtype: int64
       TCS  WIPRO
qtr1  2500   2500
qtr2  2000   2000
qtr3  3000   3000
qtr4  2000   2000
```

```
import pandas as pd
empdata={'empid':[101,102,103,104,105,106],
         'ename':['shivam','khus','jiyansh','zahir','vedant','prins'],
         'doj':['12-01-2022','1-01-2022','2-08-2012','12-08-2002','2-
05-2003','12-01-2022']
        }
df=pd.DataFrame(empdata)
print(df)
print(df.loc[0])
df.loc[0:2]
```

```
   empid    ename         doj
0    101   shivam  12-01-2022
```

```
1    102     khus   1-01-2022
2    103  jiyansh   2-08-2012
3    104    zahir  12-08-2002
4    105   vedant   2-05-2003
5    106    prins  12-01-2022
empid           101
ename        shivam
doj      12-01-2022
Name: 0, dtype: object
```

```
   empid    ename         doj
0    101   shivam  12-01-2022
1    102     khus   1-01-2022
2    103  jiyansh   2-08-2012
```

```
import pandas as pd
runs={'TCS':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000},
      'WIPRO':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000},
      'L&T':{'qtr1':2500,'qtr2':2000,'qtr3':3000,'qtr4':2000}}

df=pd.DataFrame(runs)
print(df)
print(df.iloc[0:2,1:2])
print(df.iloc[:,0:2])
```

```
       TCS  WIPRO   L&T
qtr1  2500   2500  2500
qtr2  2000   2000  2000
qtr3  3000   3000  3000
qtr4  2000   2000  2000
      WIPRO
qtr1   2500
qtr2   2000
       TCS  WIPRO
qtr1  2500   2500
qtr2  2000   2000
qtr3  3000   3000
qtr4  2000   2000
```

```
import pandas as pd
empdata={'empid':[101,102,103,104,105,106],
         'ename':['shivam','khus','jiyansh','zahir','vedant','prins'],
         'doj':['12-01-2022','1-01-2022','2-08-2012','12-08-2002','2-
05-2003','12-01-2022']
        }
df=pd.DataFrame(empdata)
print(df)
print(df.head())
print(df.tail())
```

```
   empid    ename         doj
0    101   shivam  12-01-2022
1    102     khus   1-01-2022
2    103  jiyansh   2-08-2012
3    104    zahir  12-08-2002
4    105   vedant   2-05-2003
5    106    prins  12-01-2022
   empid    ename         doj
0    101   shivam  12-01-2022
1    102     khus   1-01-2022
2    103  jiyansh   2-08-2012
3    104    zahir  12-08-2002
4    105   vedant   2-05-2003
   empid    ename         doj
1    102     khus   1-01-2022
2    103  jiyansh   2-08-2012
3    104    zahir  12-08-2002
4    105   vedant   2-05-2003
5    106    prins  12-01-2022
```

```
import pandas as pd
empdata={'empid':[101,102,103,104,105,106],
         'ename':['shivam','khus','jiyansh','zahir','vedant','prins'],
         'doj':['12-01-2022','1-01-2022','2-08-2012','12-08-2002','2-
05-2003','12-01-2022']
        }
df=pd.DataFrame(empdata)
print(df)
print(df.head(2))
print(df.tail(2))
print(df[2:5])
```

```
   empid    ename         doj
0    101   shivam  12-01-2022
1    102     khus   1-01-2022
2    103  jiyansh   2-08-2012
3    104    zahir  12-08-2002
4    105   vedant   2-05-2003
5    106    prins  12-01-2022
   empid   ename         doj
0    101  shivam  12-01-2022
1    102    khus   1-01-2022
   empid   ename         doj
4    105  vedant   2-05-2003
5    106   prins  12-01-2022
   empid    ename         doj
2    103  jiyansh   2-08-2012
3    104    zahir  12-08-2002
4    105   vedant   2-05-2003
```

```python
import pandas as pd

dic1 = {'id': ['1', '2', '3', '4', '5'], 'Value1': ['A', 'C', 'E',
'G', 'I'],
       'Value2':['B','D','F','H','J']}
dic2 = {'id': ['2', '3', '6', '7', '8'], 'Value1': ['K', 'M', 'O',
'Q', 'S'],
        'Value2': ['L', 'N', 'P', 'R', 'T']}

df1 = pd.DataFrame(dic1)
df2 = pd.DataFrame(dic2)

df3 = pd.merge(df1, df2, on='id', how='outer')

print(df3)
```

```python
import pandas as pd

# Creating dictionaries
dic1 = {'id': ['1', '2', '3', '4', '5'], 'Value1': ['A', 'C', 'E',
'G', 'I']}
dic2 = {'id': ['2', '3', '6', '7', '8'], 'Value1': ['K', 'M', 'O',
'Q', 'S'], 'Value2': ['L', 'N', 'P', 'R', 'T']}

# Creating DataFrames
df1 = pd.DataFrame(dic1)
df2 = pd.DataFrame(dic2)

# Corrected merge statement
df3 = pd.merge(df1, df2, on='id', how='outer', suffixes=('_left',
'_right'))

print(df3)
```

```python
import pandas as pd

dic1 = {'id': ['1', '2', '3', '4', '5'], 'Value1': ['A', 'C', 'E',
'G', 'I'],
       'Value2':['B','D','F','H','J']}
dic2 = {'id': ['2', '3', '6', '7', '8'], 'Value1': ['K', 'M', 'O',
'Q', 'S'],
        'Value2': ['L', 'N', 'P', 'R', 'T']}

df1 = pd.DataFrame(dic1)
df2 = pd.DataFrame(dic2)

df3 = pd.merge(df1, df2, on='id', how='inner')

print(df3)
```